

Urutan, Antrean & Prioritas

Bagaimana pekerjaan diproses: terakhir dulu, pertama dulu, paling penting dulu, atau pilihan lokal aman.

Tujuan dokumen ini: bukan menghafal tanda-tanda soal, tapi memahami gerak berpikir pattern. Setiap pattern dijelaskan dengan contoh dunia nyata, contoh teknologi, konsep komputasi, alasan kenapa contoh itu sama dengan pattern, dan template Dart minimal.

Cara Membaca Dokumen Ini

Jangan baca seperti kamus keyword. Baca seperti latihan membangun intuisi. Pattern LeetCode itu mirip alat kerja: tiap alat punya cara bergerak, situasi natural, dan bentuk data yang cocok.

Untuk tiap pattern, pakai urutan baca ini: makna manusiawi -> contoh dunia nyata -> contoh teknologi -> konsep komputasi -> bentuk LeetCode -> template Dart -> pertanyaan cek pemahaman.

Pattern	Makna inti
Stack	menyelesaikan urusan terakhir yang belum selesai
Queue	memproses urusan sesuai antrean
Heap	selalu mengambil item paling prioritas
Greedy	memilih langkah lokal yang aman untuk hasil global

Stack - menyelesaikan urusan terakhir yang belum selesai

Makna manusiawinya

Stack bekerja Last In First Out: yang terakhir masuk adalah yang pertama keluar. Ini cocok untuk urusan yang bersifat nested, undo, atau menunggu pasangan penutup.

Contoh nyata non-teknologi

Tumpukan piring

Piring terakhir ditaruh di atas, jadi paling mudah diambil dulu.

Undo di editor

Aksi terakhir adalah yang pertama dibatalkan.

Contoh nyata di dunia teknologi

Call stack program

Function terakhir yang dipanggil harus selesai dulu sebelum balik ke pemanggil.

Valid parentheses

Kurung buka terakhir harus ditutup pertama.

Konsep komputasinya

Secara komputasi, stack mendukung push dan pop di ujung. Ia menyimpan item yang belum selesai, terutama item terakhir yang paling relevan.

Kenapa contoh itu sama dengan pattern ini?

Contohnya sama karena urusan terakhir memiliki prioritas penyelesaian pertama. Stack menjaga konteks nested secara natural.

Bentuknya di LeetCode

- Valid Parentheses
- Min Stack
- Evaluate Reverse Polish Notation
- Asteroid Collision
- Monotonic Stack sebagai varian

Template Dart minimal

```
bool isValidParentheses(String s) {  
  final stack = <String>[];  
  final pairs = {')': '(', ']': '[', '}': '{'};  
  
  for (final ch in s.split('')) {  
    if (ch == '(' || ch == '[' || ch == '{') {  
      stack.add(ch);  
    } else {  
      if (stack.isEmpty || stack.removeLast() != pairs[ch]) return false;  
    }  
  }  
}
```

```
    return stack.isEmpty;  
}
```

Kalau masih bingung, tanya begini

- Apakah yang terakhir masuk harus diselesaikan dulu?
- Apakah ada struktur nested?
- Apakah gue butuh undo?

Mini drill pemahaman

1. Kenapa parentheses cocok pakai stack?
2. Apa contoh stack di browser history?
3. Apa beda stack biasa dan monotonic stack?

Queue - memproses urusan sesuai antrean

Makna manusiawinya

Queue bekerja First In First Out: yang datang dulu diproses dulu. Ini cocok untuk proses adil, batch, dan traversal level demi level.

Contoh nyata non-teknologi

Antrean kasir

Orang yang datang dulu dilayani dulu.

Antrian customer support

Ticket pertama masuk bisa diproses lebih dulu kecuali ada priority system.

Contoh nyata di dunia teknologi

Job queue

Worker mengambil job dari antrean agar proses asynchronous tetap tertib.

BFS

Queue menjaga agar node level sekarang selesai sebelum level berikutnya.

Konsep komputasinya

Secara komputasi, queue punya enqueue di belakang dan dequeue di depan. Di Dart, Queue dari `dart:collection` cocok untuk ini.

Kenapa contoh itu sama dengan pattern ini?

Contohnya sama karena semua mempertahankan urutan kedatangan. Queue bukan tentang prioritas; queue tentang fairness/urutan.

Bentuknya di LeetCode

- BFS
- Task scheduling sederhana
- Producer-consumer
- Streaming events

Template Dart minimal

```
import 'dart:collection';

void processInOrder(List<String> tasks) {
  final queue = Queue<String>.from(tasks);

  while (queue.isNotEmpty) {
    final task = queue.removeFirst();
    print('Process: $task');
  }
}
```

Kalau masih bingung, tanya begini

- Apakah urutan datang harus dihormati?

- Apakah proses menyebar level demi level?
- Apakah prioritas khusus tidak diperlukan?

Mini drill pemahaman

1. Apa bedanya queue biasa dan heap?
2. Kenapa BFS butuh queue?
3. Apa contoh queue di backend?

Heap - selalu mengambil item paling prioritas

Makna manusiawinya

Heap adalah struktur untuk selalu mengambil item terkecil/terbesar/terpenting berikutnya tanpa mengurutkan semua data setiap saat.

Contoh nyata non-teknologi

IGD rumah sakit

Pasien paling gawat ditangani dulu, bukan yang datang dulu.

Daftar tugas urgent

Dari banyak task, lu selalu ambil yang deadline paling dekat atau impact paling besar.

Contoh nyata di dunia teknologi

Priority job queue

Worker mengambil job premium atau urgent lebih dulu.

Top K analytics

Sistem mencari 10 keyword paling sering tanpa harus sort semua setiap update.

Konsep komputasinya

Secara komputasi, heap memberi push dan pop prioritas dalam $O(\log n)$. Cocok untuk top K, kth, merge k sorted, scheduling, median stream.

Kenapa contoh itu sama dengan pattern ini?

Contohnya sama karena order utama bukan kedatangan, melainkan prioritas. Heap menjaga item prioritas tetap mudah diambil.

Bentuknya di LeetCode

- Kth Largest
- Top K Frequent
- Merge K Sorted Lists
- Find Median from Data Stream
- Task Scheduler

Template Dart minimal

```
class MinHeap {
  final List<int> _data = [];

  void push(int value) {
    _data.add(value);
    int i = _data.length - 1;
    while (i > 0) {
      final p = (i - 1) ~/ 2;
      if (_data[p] <= _data[i]) break;
      final tmp = _data[p]; _data[p] = _data[i]; _data[i] = tmp;
      i = p;
    }
  }
}
```

```

int pop() {
    final result = _data.first;
    _data[0] = _data.removeLast();
    int i = 0;
    while (true) {
        int smallest = i;
        final l = 2 * i + 1, r = 2 * i + 2;
        if (l < _data.length && _data[l] < _data[smallest]) smallest = l;
        if (r < _data.length && _data[r] < _data[smallest]) smallest = r;
        if (smallest == i) break;
        final tmp = _data[i]; _data[i] = _data[smallest]; _data[smallest] = tmp;
        i = smallest;
    }
    return result;
}
}

```

Kalau masih bingung, tanya begini

- Apakah gue selalu butuh item paling kecil/besar/urgent?
- Apakah sorting penuh terlalu mahal?
- Apakah data datang streaming?

Mini drill pemahaman

1. Kenapa IGD lebih heap daripada queue?
2. Untuk top 10, kenapa min-heap ukuran 10 efektif?
3. Apa beda heap dan sorted list?

Greedy - memilih langkah lokal yang aman untuk hasil global

Makna manusiawinya

Greedy memilih keputusan terbaik saat ini yang tidak merusak masa depan. Yang sulit bukan kodenya, tapi membuktikan bahwa pilihan lokal memang aman.

Contoh nyata non-teknologi

Memilih meeting yang selesai paling cepat

Kalau ingin menghadiri meeting sebanyak mungkin, memilih meeting yang selesai paling cepat memberi ruang paling besar untuk meeting berikutnya.

Menghemat baterai saat perjalanan

Pilih mode hemat saat baterai rendah karena keputusan itu jelas tidak mengurangi kemampuan mencapai tujuan.

Contoh nyata di dunia teknologi

Interval scheduling

Sort berdasarkan end time, pilih interval yang tidak overlap.

Jump Game

Selalu simpan jangkauan terjauh yang bisa dicapai sejauh ini.

Konsep komputasinya

Secara komputasi, greedy sering menyimpan state kecil: farthest, current end, count, last chosen. Ia tidak mencoba semua kemungkinan seperti DP/backtracking.

Kenapa contoh itu sama dengan pattern ini?

Contohnya sama karena ada keputusan lokal yang bisa dianggap aman. Kalau keputusan lokal bisa menutup opsi masa depan yang penting, greedy tidak valid.

Bentuknya di LeetCode

- Jump Game
- Gas Station
- Non-overlapping Intervals
- Partition Labels
- Minimum arrows

Template Dart minimal

```
bool canJump(List<int> nums) {  
  int farthest = 0;  
  
  for (int i = 0; i < nums.length; i++) {  
    if (i > farthest) return false;  
    farthest = farthest > i + nums[i] ? farthest : i + nums[i];  
  }  
  return true;  
}
```

}

Kalau masih bingung, tanya begini

- Apa keputusan lokalnya?
- Kenapa keputusan ini aman?
- Apakah ada counterexample?
- Apakah sorting membuat pilihan lokal masuk akal?

Mini drill pemahaman

1. Buktikan kenapa memilih interval selesai paling cepat aman.
2. Kapan greedy gagal dan DP dibutuhkan?
3. Apa state kecil di Jump Game?

Penutup

Mastery bukan berarti menghafal semua tanda soal. Mastery berarti saat melihat masalah, lu bisa memodelkan gerak masalahnya: apakah ia menyebar, menyelam, menjaga window, mencari boundary, mengingat key, menggabungkan kelompok, atau menyimpan subproblem.

Cara latihan: ambil satu pattern, pahami analoginya, kerjakan 3-5 soal mudah, tulis ulang konsepnya dengan bahasa sendiri, lalu campur dengan pattern lain supaya otak belajar mengenali bentuk masalah tanpa spoiler.